PHASE 4 – IMPLEMENTATION AND DEVELOPMENT

Project Title

Railway Reservation System – Concurrent Seat Allocation Simulator

1. Introduction

Phase 4 focuses on the actual development and implementation of the Railway Reservation System using the technical specifications prepared during Phase 1, Phase 2 and Phase 3.

The primary objective is to convert the system design into a functional web application capable of managing railway seat reservations while preventing duplicate bookings during simultaneous booking requests.

The development will be carried out using Google Antigravity, an AI-powered development environment. Its coding agent will assist with generating project files, implementing application functionality, debugging errors and integrating different components.

The development process will follow a modular approach, where each component is implemented and verified before proceeding to the next.

2. Objectives

The main objectives of this phase are:

1. Develop a functional Railway Reservation System.
2. Implement a passenger-facing web interface.
3. Develop backend services using Python Flask.
4. Implement an SQLite database for storing passenger, journey, seat and booking information.
5. Develop REST APIs for communication between frontend and backend.
6. Implement seat allocation functionality.
7. Implement concurrency control to prevent double booking.
8. Integrate the frontend, backend and database.
9. Implement an administrative dashboard.
10. Prepare a working prototype for testing and verification.

3. Development Technology Stack

Technology	Purpose
Google Antigravity	AI-assisted development environment
Python	Backend programming
Flask	Web application framework
SQLite	Database management
HTML5	Frontend structure
CSS3	Interface styling
JavaScript	Frontend functionality
PyTest	Automated backend testing
Selenium	Browser automation
Git	Version control
GitHub	Source-code repository

Docker, Jenkins and SonarQube will be integrated into the development and quality workflow, with their configuration and evaluation completed in the appropriate later activities.

4. Development Activities

Step 1: Development Environment Setup

- Open Google Antigravity.
- Create a dedicated project workspace.
- Configure Python.
- Create a virtual environment.
- Install the required dependencies.
- Configure Git and GitHub.
- Verify that the development environment works correctly.

Deliverable: Configured development environment.

Step 2: Project Structure Creation

Create the folder structure according to the detailed design.

The structure will separate:

- Application logic.
- Database operations.
- Backend models.
- Frontend templates.
- Static resources.
- Testing modules.
- Documentation.

Deliverable: Complete project directory structure.

Step 3: Database Implementation

Implement the SQLite database.

Create the following tables:

- Passengers
- Journeys
- Seats
- Bookings
- Booking Requests

Implement primary keys, foreign keys, constraints and indexes.

A unique constraint must prevent multiple confirmed bookings for the same seat and journey.

Deliverable: Working database with sample records.

Step 4: Backend Model Development

Implement the primary application models:

- Passenger
- Journey
- Seat
- Booking

Each model will contain the necessary attributes and database-related operations.

Deliverable: Functional backend data models.

Step 5: Backend API Development

Develop Flask REST APIs for:

- Retrieving journeys.
- Retrieving seats.
- Checking seat availability.
- Creating bookings.
- Retrieving booking details.
- Retrieving booking history.
- Viewing administrative booking records.
- Retrieving system metrics.

Deliverable: Functional REST API endpoints.

Step 6: Seat Allocation Implementation

Implement the main seat allocation functionality.

The booking process will:

1. Receive a booking request.
2. Validate passenger information.
3. Validate journey and seat details.
4. Check seat availability.
5. Create a booking record.
6. Update seat status.

7. Return the booking result.

Deliverable: Working seat allocation functionality.

Step 7: Concurrency Control Implementation

This is the most important development activity.

Implement:

- Application-level locking for concurrent threads.
- Database transactions.
- Conditional seat-status updates.
- Database-level uniqueness constraints.
- Safe error handling and transaction rollback.

Demonstrate the situation where two passengers attempt to book the last available seat.

Only one passenger must receive a confirmed booking.

Deliverable: Working concurrency-controlled booking mechanism.

Step 8: Frontend Development

Develop the following interfaces:

1. Passenger Dashboard.
2. Journey Search Page.
3. Seat Selection Interface.
4. Booking Confirmation Page.
5. Booking History Page.
6. Admin Dashboard.

The interface should be responsive, simple and easy to navigate.

Deliverable: Functional frontend.

Step 9: Frontend and Backend Integration

Connect the frontend to the Flask APIs.

Implement:

- API communication.
- Dynamic seat availability.
- Booking request submission.

- Booking confirmation.
- Error messages.
- Data retrieval.
- Admin dashboard updates.

Deliverable: Fully integrated web application.

Step 10: Working Prototype Completion

Run the application and verify the main booking workflow.

Check:

- Application startup.
- Database connectivity.
- Journey retrieval.
- Seat availability.
- Booking creation.
- Booking rejection.
- Concurrent booking behaviour.
- Frontend-backend communication.

Fix implementation defects and prepare screenshots.

Deliverable: Working Railway Reservation System prototype.

5. Expected System Features

The completed prototype should provide:

Feature ID	Feature	Description
F-01	Passenger Interface	Allows passengers to access the system
F-02	Journey Search	Allows users to search journeys
F-03	Seat Availability	Displays available and booked seats

F-04	Seat Selection	Allows passengers to select seats
F-05	Seat Booking	Processes booking requests
F-06	Concurrency Control	Prevents simultaneous duplicate allocation
F-07	Booking Confirmation	Displays booking results
F-08	Booking History	Displays passenger booking records
F-09	Admin Dashboard	Displays booking information
F-10	System Monitoring	Displays booking and error metrics

6. Development Approach

The project will be developed incrementally using Google Antigravity.

The development workflow will be:

1. Provide the agent with project requirements.
2. Ask it to inspect the existing workspace.
3. Generate the required implementation.
4. Review the generated code.
5. Execute the application.
6. Identify and fix errors.
7. Verify the completed functionality.
8. Commit stable changes to GitHub.

Each development milestone must be completed and verified before starting the next.

7. Expected Deliverables

At the end of Phase 4, the following deliverables should be available:

- Complete project source code.
- Functional Flask application.
- SQLite database.
- REST API implementation.
- Seat allocation algorithm.
- Concurrency control implementation.
- Passenger-facing frontend.
- Administrative dashboard.
- Frontend-backend integration.
- Working local prototype.
- GitHub repository.
- Implementation screenshots.
- Development documentation.

8. Success Criteria

Phase 4 will be considered complete when:

1. The application starts successfully.
2. The database connects correctly.
3. Passenger and journey records can be retrieved.
4. Seat availability is displayed correctly.
5. A passenger can book an available seat.
6. A passenger cannot book an already-booked seat.
7. Two simultaneous requests for the same last seat cannot both succeed.
8. Booking records are stored correctly.
9. The frontend communicates successfully with the backend.
10. The administrative dashboard displays booking information.

The 50,000 booking attempts per minute and 95th-percentile response time below two seconds are performance targets that must be evaluated separately during Phase 5.

9. Conclusion

Phase 4 transforms the Railway Reservation System design into a functional software prototype.

Google Antigravity will be used to accelerate development, while the implementation will be reviewed and verified through actual execution.

Special attention will be given to the seat allocation and concurrency control mechanisms because preventing duplicate confirmed bookings is the primary requirement of the project.

The completed prototype will provide the foundation for Phase 5 – Testing and Verification.